# 训练的本质

- 来源: https://class.imooc.com/lesson/5253#mid=104871
- 阶段: 阶段一: 【认知基石】理解大模型的本质与工作原理
- 周: 第1周 语言模型是什么
- 模块: ChatGPT的本质
- 课节: 图文 1-5 训练的本质

## 1-5 训练的本质

## 训练的本质

## 2.1 训练的本质：迭代逼近，而非直接求解

既然不能直接解方程组，那就换个思路——用海量数据进行训练。

训练不是直接解出参数，而是通过逐步调整，逐渐逼近最优解。

打个比方：

解方程组像用GPS直接定位——一步到位，但需要完整的地图和精确计算。

训练则像在黑暗中摸索——每走一步感受一下方向对不对，对了就继续，错了就调整，最终也能到达目的地。

## 2.2 训练的完整流程：从混沌到有序

为了让你直观地理解训练的完整过程，我先用流程图展示这个循环：

这个图展示了训练的核心循环（关于训练我们在后续章节会讨论），而这个循环会重复数千亿次，每次都让模型变得更聪明一点点，直到最终"智能涌现"（关于涌现，我们在后面章节会讨论）。

现在让我用一个具体的例子，带你看看这个从混沌到有序的神奇过程。

> [!TIP] 阅读下文时，请结合上述流程图，效果更佳。

> [!TIP]

阅读下文时，请结合上述流程图，效果更佳。

- 步骤一：随机初始化时的混乱状态

训练开始的那一刻，这1750亿个参数都是随机的小数，通常在-0.1到0.1之间。这些数字毫无意义，就像一堆完全随机的噪音。此时的ChatGPT，对语言一无所知。

如果你问它：“你好吗？” 它可能输出："@#$%^&*哈哈哈哈的的的的是是是" —— 完全的胡言乱语。

- 步骤二：第一次学习——预测"今天天气"的下一个词

训练正式开始。我们给它喂第一个文本片段："今天天气" 。模型的任务：预测下一个词。

在初始的混乱状态下，由于参数是随机的，它可能给出这些候选：

> “电脑” 概率：8% “苹果” 概率：12% “真好” 概率：3% ← 这才是合理的，但概率最低！
> “紫色” 概率：7% ...

模型可能选择"苹果"（概率最高）作为下一个词。但这显然和天气不搭边。完全错误！

> [!NOTE] 为什么预测的是下一个词？而不是整个完整句子？ 这个我们后面章节会探讨，这里暂时这样记住。 为什么是词的概率？ 模型预测的不是一个确定的词，而是每个词出现的可能性（概率）。这个机制我们会在下一节"文字接龙游戏"中详细讲解。现在你只需要知道：概率越高，模型越倾向于选择这个词。

- 步骤三：计算错误——模型意识到自己错了

我们告诉它正确答案：“真好” 。模型计算：我错了！我应该提高"真好"的概率，降低"苹果"的概率。我们用"损失"(Loss)来衡量错误的大小。

> [!NOTE] 什么是Loss（损失）？ Loss是一个数字，衡量"预测值"和"正确答案"之间的差距。差距越大，Loss越大。训练的目标就是让Loss越来越小，最终接近0。 这个思想不仅可以用来训练，还可以用来训练现在流行的智能体，大概思路是，让B智能体给A智能体打分；设定某个分数阈值，比如90分，A不断地调整，一直到达到90分。

- 步骤四：调整参数

关键来了：模型怎么改进？

模型会分析："我预测错误，是因为哪些参数设置得不对？"然后调整这些参数。

调整前：

> 参数$_{12345}$ = 0.0234 参数$_{67890}$ = -0.0567

调整后：

> 参数$_{12345}$ = 0.0234 + 0.0001 = 0.0235 ← 增加了一点点 参数$_{67890}$ = -0.0567 - 0.0001 = -0.0568 ← 减少了一点点

这个调整非常微小，但方向是对的：让模型下次预测"真好"的概率更高一点。

> [!TIP] 这里有个问题，模型怎么知道该增加还是减少？怎么知道调多少？这背后有一套完整的数学机制，我们会在后面详细讲解。现在只需要记住：模型有办法精确计算每个参数该怎么调，然后每次微调一点点。

- 步骤五：累积的奇迹——数千亿次迭代后

这个过程不断重复：

> 读第1个句子 → 预测错了 → 调整参数 读第2个句子 → 预测还是错了 → 再调整参数 读第3个句子 → 预测错误减少了一点 → 继续调整 ... 读第1000个句子 → 预测准确率从0%提升到了5% ... 读第100万个句子 → 准确率提升到30% ... 读第10亿个句子 → 准确率提升到60% ... 读第3000亿个词 → 准确率稳定在75%左右

每一次调整都很微小，但累积起来，那1750亿个参数慢慢地、慢慢地，从随机的噪音，变成了有意义的知识表示。

# 2.3 训练背后的机制：反向传播、梯度下降与学习率

前面我们讲了训练的流程，现在让我们深入理解：模型到底是如何知道该调哪些参数、往哪个方向调、调多少的?

这涉及三个核心机制：反向传播、梯度下降和学习率。

## 反向传播：找出该调哪些参数

模型预测错了，但有1750亿个参数，该调哪些?

反向传播的解决方案是：从错误结果往回推，追溯责任链。

想象你在玩飞镖。你投了一镖，偏左了。你需要分析：

结果：偏左了

往回分析：

```
← 是手腕角度不对？（负责5%的偏差）
← 还是手臂力度不对？（负责30%的偏差）
← 还是站位不对？（负责65%的偏差）
```

反向传播就是这个分析过程——从错误结果往回推，计算出每个参数对错误的"责任大小"。

> [!NOTE] 具体怎么算？ 这涉及微积分中的链式法则。简单来说，如果A影响B，B影响C，那么A对C的影响等于A对B的影响乘以B对C的影响。模型中每个参数通过多层网络影响最终输出，链式法则可以从输出层往回逐层相乘，算出每个参数的精确贡献。 对于大模型应用开发或者微调大模型，不需要深入理解这个数学过程，知道原理即可。

在ChatGPT训练中，当预测出"苹果"（错误），我们告诉ChatGPT正确答案是"真好"，那么ChatGPT会进行反向传播计算：

```
← 参数12345的梯度 = +2.5
← 参数67890的梯度 = -1.8
← 参数11111的梯度 = 0
```

这样，模型就知道了：参数$_{12345}$责任最大（梯度绝对值最大），要重点调整。

> [!NOTE] 什么是梯度？ 梯度就是反向传播计算出来的数字，表示参数对Loss的影响程度。你可以理解为"责任值"——绝对值越大，责任越大。

## 梯度下降：让参数往正确的方向调整

反向传播告诉我们哪些参数需要调，梯度下降负责执行调整。

梯度下降的核心思想很简单：顺着能让错误变小的方向，调整参数。

```
参数新值 = 参数旧值 - 学习率 × 梯度


1
```

这个公式自动保证了调整方向的正确性——无论梯度是正还是负，参数都会往让Loss下降的方向走。你不需要手动判断该调大还是调小，公式里的减号会处理好一切。 这就是"梯度下降"的含义：沿着梯度的反方向，让错误不断下降。

## 学习率：控制调整幅度

学习率决定了每次调整的幅度。太大或太小都不好：

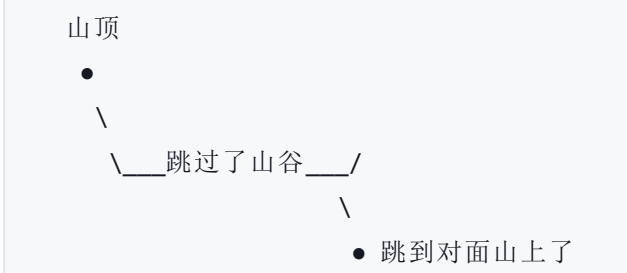
实际例子：

> 当前值：参数$_{12345}$ = 0.0234 梯度：+2.5（反向传播计算的） 学习率：0.0001（人工设定的） 计算新参数值： 参数新值 = 0.0234 - 0.0001 × 2.5 = 0.0234 - 0.00025 = 0.0233975 验证效果：把新参数代入模型重新计算，Loss 从 10.5 降到了 10.48

学习率的权衡：

学习率的权衡

学习率太大（比如 0.1）




问题：步子太大，跳过最优解
训练不稳定，甚至发散

学习率太小（比如 0.0000001）




问题：训练极其缓慢，时间成本无法接受

```
学习率合适（比如 0.0001）
————————————————————————————

  山顶 Loss=100
   ● ← 第1次
    \
     ● ← 第100次 Loss=85
      \
       ● ← 第1000次 Loss=60
        \  ← 每次都有明显下降
         ● ← 第1万次 Loss=20
          \
           ● ← 第10万次 Loss=2（山谷）

  效果：稳定收敛、速度合理
```

## 总结

训练的本质可以用一句话概括：以降低Loss为目的，通过反向传播和梯度下降手段，用数据驱动的方式迭代逼近最优解。

目的：最小化Loss（预测值与正确答案的差距）

手段： • 反向传播 —— 计算每个参数的梯度（责任大小） • 梯度下降 —— 沿梯度反方向调整参数，确保Loss下降

方法： • 参数更新公式：参数新值 = 旧值 - 学习率 × 梯度 • 学习率控制 —— 决定每次调整的幅度

策略： • 学习率动态衰减 —— 初期大步快跑，后期精细调整 • 海量数据迭代 —— 数千亿次微调，量变引发质变

## 学习率的动态调整

训练GPT-3时，研究人员不会从头到尾使用同一个学习率，而是采用动态调整：

动态调整

> 训练初期：学习率 = 0.001（较大） 参数还很随机，需要快速调整，像下山时在山腰大步走 训练中期：学习率 = 0.0001（适中） 已经接近最优解，需要稳步前进 训练后期：学习率 = 0.00001（很小） 已经很接近山谷，需要精细调整，像在山谷里小心翼翼找最低点

这就是"学习率衰减"策略：开始用大步子快速下山，越接近山谷步子越小，确保不会跳过最优解。

## 2.4 完整流程：从错误到优化

让我用一个完整的例子展示参数更新的全过程：

```
预测："苹果"（错误）
正确："真好"
Loss = 10.5
```

- 反向传播：计算梯度

```
参数12345：梯度 = +2.5
参数67890：梯度 = -1.8
参数11111：梯度 = 0（这次错误跟它无关，不用调）
...
（一次性计算出所有1750亿个参数的梯度）
```

- 应用公式：更新参数

公式：参数新值 = 参数旧值 - 学习率 × 梯度，学习率 = 0.0001

```
参数12345：0.0234 - 0.0001 × 2.5  = 0.0233975
参数67890：-0.0567 - 0.0001 × (-1.8) = -0.0565200
参数11111：0.1234 - 0.0001 × 0 = 0.1234（保持不变）
...
（每次训练只更新梯度≠0的参数）
```

```
用新参数重新预测："这部电影___"
预测："很好看"
正确："一般般"
Loss = 9.2（比10.5小，说明有进步！）
```

- 重复亿万次

每次用不同的训练数据，逐步优化所有参数：

```
Loss = 10.5 → 9.2 → 8.7 → ... → 2.5


1
```

## 2.5 从精确求解到数据驱动：这是一种哲学意义上的转变

这个从"解"到"训练"的转变，不仅仅是技术细节，而是一个哲学上的根本转变。

传统方法追求精确的数学解。 用传统方法数学公式描述规律，通过解方程得到精确答案。这需要我们完全理解问题的本质，所以只适合简单、可解释的问题。

深度学习方法用海量数据堆出智能。 不再追求精确的数学解，而是用数十亿、数千亿的数据样本，让模型在海量数据中"自己悟出"规律。

这是一种量变引发质变的过程——当数据量足够大、参数量足够多，智能会自己涌现出来。

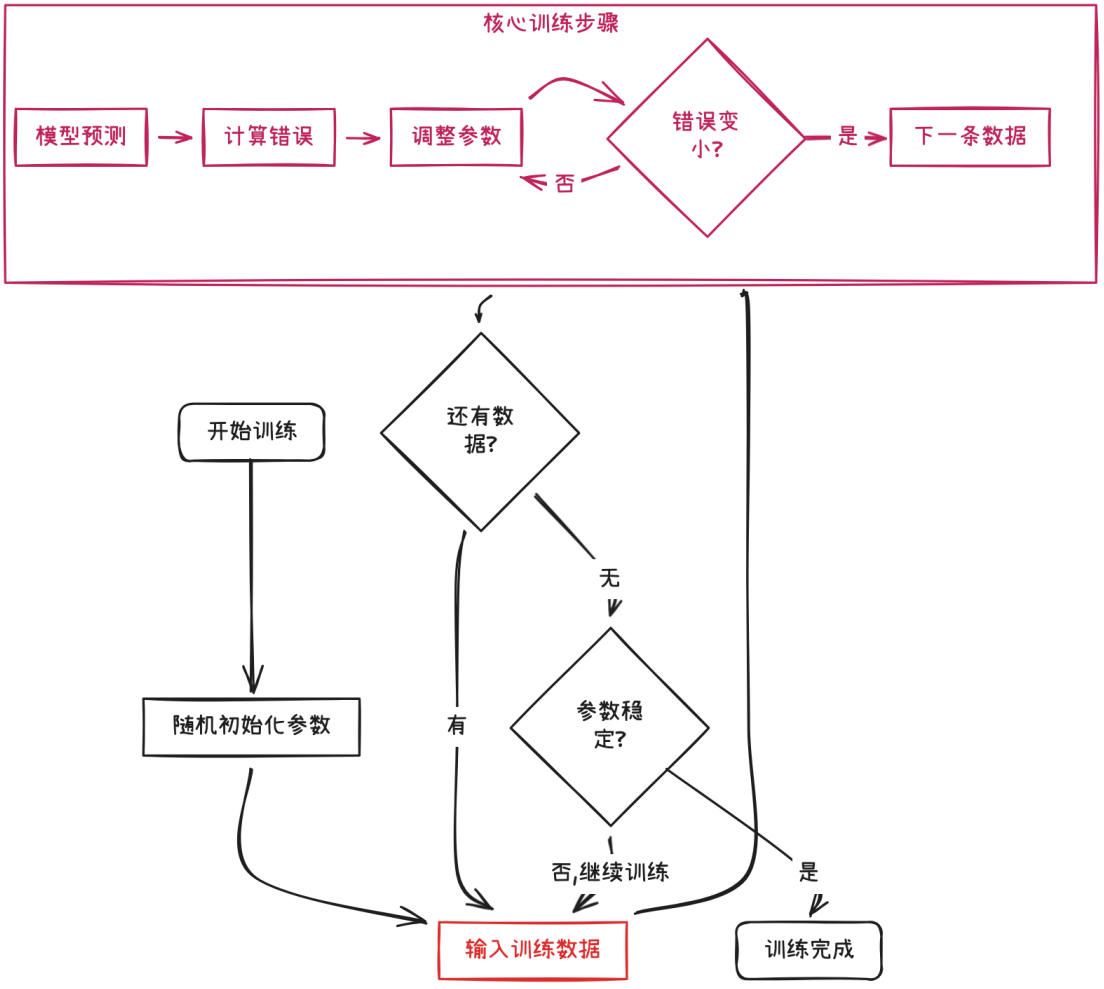
我们不再试图用数学公式精确描述“语言是什么”，而是相信: 当数据量足够大，参数量足够多，智能会自己涌现出来。。这个思想，其实和早些年流行的"大数据"有异曲同工之妙，都是用有点暴力的方式，靠统计规律来推出结果。

这，就是训练的本质。

## 2.6 ■ 学点英语

| 中文 | English | 音标 | 说明 |
| --- | --- | --- | --- |
| 训练 | Training | /ˈtreɪnɪŋ/ | 通过数据迭代优化模型参数 |
| 参数 | Parameter | /pəˈræmɪtər/ | 模型可学习的权重和偏置 |
| 损失 | Loss | /lɒs/ | 预测值与正确答案的差距 |
| 随机初始化 | Random Initialization | /ˈrændəm ɪˌnɪʃəlaɪˈzeɪʃən/ | 参数在训练前设为随机小数 |
| 反向传播 | Backpropagation | /ˌbækˌprɒpəˈgeɪʃən/ | 从输出层往回计算参数的梯度 |
| 梯度 | Gradient | /ˈgreɪdiənt/ | 参数对Loss影响程度的数值 |
| 梯度下降 | Gradient Descent | /ˈgreɪdiənt dɪˈsent/ | 沿Loss下降方向迭代调整参数 |
| 学习率 | Learning Rate | /ˈlɜːnɪŋ reɪt/ | 控制参数调整幅度的超参数 |
| 学习率衰减 | Learning Rate Decay | /ˈlɜːnɪŋ reɪt dɪˈkeɪ/ | 训练初期大步、后期精细调整 |
| 迭代 | Iteration | /ˌɪtəˈreɪʃən/ | 一次前向+反向+参数更新的全过程 |
| 时期 | Epoch | /ˈiːpɒk/ | 完整遍历一次整个训练数据集 |
| 批量 | Batch | /bætʃ/ | 一次更新中使用的数据子集 |
| 数据驱动 | Data-Driven | /ˈdeɪtə ˈdrɪvən/ | 用海量数据而非数学精确解的方法论 |
| 链式法则 | Chain Rule | /tʃeɪn ruːl/ | 反向传播的微积分数学基础 |

# 图片


核心训练步骤
模型预测
计算错误
调整参数
错误变小?
否
是
下一条数据
开始训练
还有数据?
无
随机初始化参数
有
参数稳定?
否,继续训练
是
输入训练数据
训练完成